

Roadmap complet Java + Spring Boot + Kafka

Curriculum riguros de la fundație la producție · 40 de săptămâni



Scopul documentului

Acest curriculum descrie ce trebuie să învețe un inginer care vrea să lucreze profesionist pe **Java + Spring Boot + Apache Kafka**: de la limbaj, JVM și baze de date până la API-uri, event streaming, consistență, securitate, testare, observabilitate și operare în producție.

Ținta nu este memorarea adnotărilor. La final trebuie să poți lua o cerință ambiguă, să alegi o arhitectură potrivită, să implementezi sistemul, să demonstrezi corectitudinea lui și să-l operezi când apar trafic, mesaje duplicate, rebalansări, erori parțiale și schimbări de schemă.

Documentul a fost cercetat în august 2026. Recomandările sunt ancorate în documentații oficiale Java/OpenJDK, Spring, Apache Kafka și ale proiectelor folosite, marcate [S1]–[S44]. Bibliografia se află la final.

Profilul profesional urmărit

Un rol solid de Java/Spring/Kafka combină cinci discipline:

Axă	Ce trebuie demonstrat	Pondere orientativă
Java și JVM	Limbaj, colecții, concurență, memorie, profiling	20%
Backend Spring	API-uri, DI, tranzacții, securitate, configurație	25%
Date	SQL, PostgreSQL, JPA/Hibernate, migrații, locking	20%
Kafka și EDA	Producer/consumer, ordering, delivery, schemas, Streams	25%
Producție și product	Teste, observabilitate, cloud, incidente, stakeholderi	10%

Rezultatul urmărit

La final trebuie să poți spune, cu dovezi în portofoliu:

- pot implementa un serviciu Spring Boot modular, securizat și testabil;
- pot explica ce garantează o tranzacție SQL și ce nu poate garanta peste două sisteme;
- pot alege cheia, numărul de partiții și contractul unui eveniment Kafka;
- pot proiecta produceri idempotenți și consumatori toleranți la duplicate;
- pot gestiona retry, backoff, DLT, poison messages și reprocessing;
- pot testa cu PostgreSQL și Kafka reale în containere;
- pot urmări un request și un eveniment prin logs, metrics și traces;
- pot diagnostica N+1, consumer lag, rebalance storms, GC pressure și timeouts;
- pot livra gradual, cu rollback, migrații sigure și runbook-uri;
- pot explica stakeholderilor consistența, riscurile, costul și compromisurile.

Cum folosești curriculumul

Durată și ritm

Planul de bază este de **40 de săptămâni**, la 12–15 ore pe săptămână. Un programator Java cu experiență poate comprima primele șase săptămâni, dar nu ar trebui să sară peste testarea distribuită, Kafka correctness sau operarea în producție.

O săptămână tipică:

- 3 ore documentație și concepte;
- 7–9 ore implementare;
- 2 ore teste și experimente de defectare;
- 1 oră documentare: ADR, diagrame, runbook sau postmortem;
- 1 oră recapitulare și explicație orală.

Regula de promovare

Un subiect este învățat doar când ai patru dovezi:

- 1 **Explicație:** îl poți explica în limbaj simplu și poți descrie limitele.
- 2 **Implementare:** îl poți construi fără copy-paste orb.
- 3 **Măsurare:** poți alege metrice și praguri utile.
- 4 **Diagnostic:** poți reproduce și izola o defecțiune.

Nu avansezi fiindcă ai terminat un curs. Avansezi când treci gate-ul practic al capitolului.

Diagnostic inițial

Înainte de plan, încearcă fără tutorial:

- un API CRUD cu validare, erori coerente și autentificare;

- o interogare SQL cu join, index și **EXPLAIN ANALYZE**;
- o tranzacție care previne două actualizări concurente incompatibile;
- un test JUnit unit și unul de integrare cu bază de date reală;
- un producer și un consumer Kafka cu cheie și headers;
- o explicație pentru offset, consumer group și rebalance;
- o imagine Docker non-root și un endpoint de readiness;
- un diagnostic simplu cu thread dump sau Java Flight Recorder.

Punctele ratate devin obligatorii; punctele reușite pot fi parcurse accelerat.

Capitolul 1 — Java modern, fără goluri în fundație

1.1 Limbajul

Stăpânește:

- tipuri primitive, referințe, autoboxing și egalitate;
- control flow, methods, varargs și scope;
- clase, interfețe, enum-uri și adnotări;
- records pentru value objects și sealed classes pentru ierarhii închise;
- generics, wildcards, invariance și regula PECS;
- excepții checked/unchecked și o taxonomie coerentă a erorilor;
- immutability, defensive copies și obiecte valide din construcție;
- **Optional** pentru absența unei valori returnate, nu pentru orice câmp;
- date/time cu **Instant**, **LocalDate**, **ZonedDateTime** și **Duration**;
- pattern matching și expresii **switch** moderne.

Învață diferența dintre identitatea unui obiect și valoarea lui. Contractele **equals/hashCode** sunt esențiale pentru colecții, cache, entități și deduplicare.

Exercițiu: modelează comenzile unui magazin folosind records pentru identificatori și bani, sealed interfaces pentru stări și validare la construcție. Scrie teste pentru egalitate și invariants.

1.2 Colecții și algoritmi

Collections Framework definește contracte și implementări pentru liste, seturi, cozi și map-uri [S2]. Trebuie să știi:

- complexitatea uzuală pentru **ArrayList**, **HashMap**, **TreeMap** și **ArrayDeque**;
- efectul coliziunilor și al unui **hashCode** prost;
- ordering, **Comparable**, **Comparator** și stabilitatea sortării;
- immutable/unmodifiable collections;

- iterators și fail-fast behavior;
- alegerea dintre set, listă și map pe baza invariantelor, nu a comodității;
- algoritmi de bază: căutare, sortare, heap, BFS/DFS și sliding window.

Nu trebuie să devii concurent de olimpiadă. Trebuie să recunoști când un endpoint face $O(n^2)$, când încarcă inutil toate rândurile și când structura de date încalcă modelul domeniului.

1.3 Streams și programare funcțională

Învăță **Predicate**, **Function**, **Supplier**, method references și compoziție. Pentru Streams:

- operații intermediare versus terminale;
- lazy evaluation, short-circuiting și side effects;
- **map**, **flatMap**, **filter**, **reduce**, collectors și grouping;
- costul boxing-ului și al pipeline-urilor greu de citit;
- de ce parallel streams nu sunt o optimizare implicită;
- când un loop este mai clar.

Regulă: stream-urile exprimă transformări. Nu ascunde I/O, tranzacții sau mutații surprinzătoare într-un **map**.

1.4 I/O, serializare și networking

Învăță:

- bytes versus text și UTF-8;
- **Path**, **Files**, streams, readers/writers și try-with-resources;
- buffering și procesarea incrementală a fișierelor mari;
- JSON cu Jackson: naming, unknown fields, date/time și custom serializers;
- sockets și HTTP la nivel conceptual;
- timeouts de connect/read/write;
- DNS, TLS și connection pooling.

Gate 1

Construiește o bibliotecă Java fără Spring care citește un fișier mare de tranzacții, validează și agregă incremental, expune erori structurate și are minimum 85% branch coverage pe logica de domeniu. Compară memoria și timpul pentru o variantă streaming și una care încarcă totul.

Capitolul 2 — Build, testare și calitatea codului

2.1 Maven sau Gradle

Învăță bine unul și citește suficient din celălalt:

- lifecycle/tasks, dependency scopes/configurations și plugins;

- dependency management, BOM și conflict resolution;
- reproducible builds și dependency locking;
- multi-module builds;
- separarea unit/integration tests;
- annotation processors;
- verificări de formatare, static analysis și vulnerabilități în CI.

În proiectele Spring Boot, lasă BOM-ul framework-ului să gestioneze versiunile compatibile. Nu suprascrie versiuni individuale fără motiv și test de compatibilitate.

2.2 JUnit, AssertJ și test doubles

Stăpânește:

- test lifecycle, nested tests și parameterized tests [S34];
- assertions lizibile și mesaje utile;
- fakes, stubs, spies și mocks;
- Mockito pentru colaboratori externi, nu pentru fiecare obiect;
- determinism: clock injectat, UUID generator și random seed;
- property-based tests pentru invariants;
- mutation testing ca audit opțional al calității testelor.

O suită bună descrie comportamentul observabil și rezistă refactorizării. Dacă testul reproduce linie cu linie implementarea, are valoare mică.

2.3 Design și mentenanță

Învăță:

- cohesion, coupling și information hiding;
- SOLID ca set de întrebări, nu ca religie;
- composition over inheritance;
- ports and adapters / arhitectură hexagonală;
- dependency rule și separarea domeniului de infrastructură;
- package by feature;
- refactoring incremental și characterization tests;
- ADR-uri pentru decizii cu alternative și consecințe.

Semnale de alarmă: servicii de mii de linii, entități anemice plus logică împrăștiată, clase **Utils**, adnotări framework în nucleul domeniului și constructori cu zece dependențe.

Gate 2

Creează un proiect multi-module cu **domain**, **application**, **infrastructure** și **bootstrap**. Domeniul trebuie testat fără Spring. Aduă CI care compilează, rulează unit și integration tests, verifică stilul și produce artefactul.

Capitolul 3 — Concurență, Java Memory Model și JVM

3.1 Concurență corectă

Pachetul **java.util.concurrent** oferă executors, queues, locks, atomics și synchronizers [S3]. Învață:

- process, platform thread și virtual thread;
- race condition, visibility, atomicity și ordering;
- happens-before și publicare sigură;
- **synchronized**, **volatile**, **Lock** și atomics;
- concurrent collections și blocking queues;
- thread pools, queue capacity, rejection și starvation;
- **CompletableFuture**, compoziție, timeout și propagarea erorii;
- cancellation și interrupt; nu înghiți **InterruptedException**;
- semaphores pentru limitarea unei resurse rare.

3.2 Virtual threads

Virtual threads au devenit funcționalitate finală în Java 21 și sunt potrivite pentru throughput mare în sarcini cu mult I/O blocant [S4]. Ele nu fac munca CPU-bound mai rapidă și nu trebuie „pooled” ca platform threads. Limitează explicit accesul la DB sau API-uri prin pool-uri de conexiuni și semaphores.

Învață să măsoari:

- throughput și latență p50/p95/p99;
- numărul de cereri simultane;
- pool saturation;
- pinning/blocking problematic;
- efectul timeout-urilor și al backpressure-ului.

Structured concurrency grupează task-uri înrudite și coordonează join, cancel și error handling, dar în JDK 25 rămâne preview; trateaz-o ca subiect avansat, nu ca bază stabilă de producție [S5].

3.3 Memorie și garbage collection

Învață:

- stack, heap, metaspace și direct buffers;
- reachability și surse comune de memory leak;
- allocation rate, live set și GC pause;
- collectors principali și obiectivele latency/throughput;
- **-Xms, -Xmx**, container awareness și headroom;
- thread dumps, heap dumps și Native Memory Tracking;
- de ce tuning-ul începe după măsurare. Ghidul oficial Java 25 prezintă collectorii și criteriile de alegere [S6].

3.4 Profiling și Java Flight Recorder

JFR poate arăta CPU, allocations, GC, threads, locks și I/O, cu overhead potrivit și pentru diagnostic în producție [S7]. Exersează:

- o înregistrare temporizată;
- identificarea hot methods și allocation hotspots;
- diferența dintre CPU saturation și lock contention;
- corelarea unei degradări cu GC sau I/O;
- un before/after bazat pe aceeași încărcare.

Gate 3

Construiește un server care apelează trei servicii lente. Compară platform threads, un pool limitat și virtual threads. Injectează timeouts, anulări și 2.000 de cereri concurente. Predă grafice p95/p99, JFR, explicația limitelor și o decizie argumentată.

Capitolul 4 — Spring Core și Spring Boot

4.1 IoC și dependency injection

Înțelege containerul Spring, nu doar adnotările:

- bean definitions, component scan și Java configuration;
- constructor injection, preferată pentru dependențe obligatorii și obiecte complet inițializate [S8];
- bean lifecycle, scopes și proxies;
- **@Configuration, @Bean, @Component** și stereotypes;
- conditional beans și profiles;
- circular dependencies ca simptom de design;
- AOP pentru preocupări transversale, cu atenție la self-invocation.

Scrie clase care pot fi instantiate în teste fără context Spring. Framework-ul trebuie să compună aplicația, nu să devină modelul ei de domeniu.

4.2 Auto-configuration și configurare externă

Învăță:

- starters și auto-configuration reports;
- precedence pentru properties/YAML/environment/command line;
- **@ConfigurationProperties** tipizat și validat;
- secrets în secret manager, nu în repository;
- profile-uri puține și diferențe explicite între medii;
- fail-fast pentru configurație invalidă;
- graceful shutdown.

În august 2026, Spring Boot 4.1.0 cere cel puțin Java 17 și este compatibil până la Java 26 [S9]. Pentru învățare folosește un JDK LTS modern, de preferat 21 sau 25, și matricea de compatibilitate a proiectului. Nu confunda „ultimul număr” cu o migrare sigură.

4.3 Structură modulară

Începe cu un modular monolith înainte să distribui accidental complexitatea. Spring Modulith poate deriva modulele din packages, verifica dependențe, genera documentație și testa module izolat [S10][S11].

Un modul bun are:

- API public mic;
- implementare internă ascunsă;
- model și tranzație coerente;
- evenimente de domeniu explicite;
- teste proprii;
- dependențe într-o singură direcție.

Gate 4

Construiește un Spring Boot modular pentru catalog, comenzi și inventar. Verifică automat lipsa dependențelor ciclice. Fiecare modul trebuie să poată fi testat separat, iar configurația invalidă trebuie să oprească pornirea cu un mesaj clar.

Capitolul 5 — HTTP, REST și contracte API

5.1 Semantica HTTP

Învăță:

- metode safe și idempotent;
- status codes corecte;
- cache headers, ETag și conditional requests;
- content negotiation;

- pagination, filtering și sorting;
- idempotency keys pentru operații sensibile;
- request size limits și streaming;
- timeouts și anulare.

Un endpoint **POST** poate fi făcut idempotent la nivel de business printr-o cheie unică stocată tranzacțional. Asta nu îl transformă semantic în **PUT**, dar previne efectele duble la retry.

5.2 Spring MVC

Stăpânește:

- **@RestController**, routing și argument resolution;
- DTO-uri separate de entitățile JPA;
- Bean Validation și validări cross-field [S12];
- Jackson configuration;
- **@ControllerAdvice** pentru tratarea transversală a excepțiilor [S13];
- **ProblemDetail** și răspunsuri conforme RFC 9457 [S14];
- filtre, interceptors și correlation IDs;
- OpenAPI și contract-first unde interfețele sunt partajate.

5.3 Compatibilitate și evoluție

Învăță să adaugi câmpuri opționale, să păstrezi semantica câmpurilor existente și să folosești deprecarea înainte de eliminare. Pentru schimbări incompatibile, alege explicit versionarea în URL, header sau media type și documentează perioada de migrare.

Gate 5

Livrează un API de comenzi cu validare, Problem Details, pagination, optimistic concurrency prin version/ETag și idempotency key. Testează contractul, duplicate requests și două actualizări concurente.

Capitolul 6 — SQL, PostgreSQL și tranzacții

6.1 SQL înainte de ORM

Învăță:

- modelare, chei primare/străine și constraints;
- normalizare și denormalizare intenționată;
- joins, subqueries, CTE și window functions;
- indexes B-tree și composite; ordinea coloanelor;
- selectivity, cardinality și query planner;
- **EXPLAIN (ANALYZE, BUFFERS);**

- pagination cu cursor versus offset;
- connection pooling și limitele DB.

PostgreSQL recomandă examinarea planurilor și statisticilor reale pentru a verifica folosirea indexurilor [S15]. Nu adăuga indexuri după intuiție: fiecare index costă spațiu și scrieri.

6.2 ACID, isolation și locking

Stăpânește:

- atomicity, consistency, isolation și durability;
- lost update, dirty/non-repeatable/phantom reads;
- isolation levels și comportamentul concret al PostgreSQL;
- optimistic locking pentru conflicte rare;
- pessimistic locking pentru secțiuni critice justificate;
- unique constraints ca ultimă linie împotriva dublurilor;
- deadlocks, ordering consistent și retry limitat.

Tranzacția este o graniță de consistență. Nu ține o tranzacție SQL deschisă în timp ce aștepti un API lent sau publici sincron într-un broker.

6.3 Migrații

Folosește Flyway sau Liquibase. Flyway aplică migrațiile versionate în ordine și urmărește versiunea/checksum-ul în schema history [S16]. Practici:

- nu modifica o migrare aplicată într-un mediu permanent;
- preferă roll-forward;
- separă schimbările expand și contract;
- backfill în batch-uri;
- creează indexuri fără blocaje când platforma permite;
- testează migrarea pe volum realist;
- stabilește cine rulează migrarea la deployment.

Gate 6

Modelează inventarul cu rezervări concurente. Demonstrează prin teste că stocul nu devine negativ. Compară optimistic și pessimistic locking, provoacă un deadlock controlat și documentează retry-ul sigur.

Capitolul 7 — JPA, Hibernate și Spring Data

7.1 Modelul de persistență

Jakarta Persistence standardizează ORM-ul și ciclul de viață al entităților [S17]. Învăță:

- entity states: transient, managed, detached, removed;

- persistence context și identity map;
- dirty checking și flush;
- mapping pentru value objects și aggregates;
- relații, owning side și cascade;
- lazy/eager loading și granița tranzacției;
- entity equality și identificatori generați;
- auditing și soft delete doar când cerința o cere.

7.2 Query performance

Hibernate documentează fetch strategies, batching, locking și flush [S18]. Stăpânește:

- N+1 și detectarea lui în teste/metrics;
- fetch join, entity graphs și batch fetching;
- DTO projections pentru read models [S19];
- pagination fără fetch join periculos peste colecții;
- bulk updates și efectul asupra persistence context;
- JDBC batching;
- query hints doar după măsurare.

Repository-urile reduc boilerplate, dar nu înlocuiesc modelarea query-urilor. Pentru rapoarte complexe, SQL/jOOQ/JdbcClient poate fi mai transparent decât forțarea ORM-ului.

7.3 Tranzacții Spring

Spring oferă o abstracție consistentă pentru tranzacții declarative și programatice [S20].
Învăță:

- **@Transactional**, propagation și isolation;
- rollback rules;
- proxy boundaries și problema self-invocation;
- read-only ca hint, nu control absolut;
- timeout;
- transaction synchronization;
- testele care pot ascunde lazy-loading sau rollback behavior.

Gate 7

Implementează query-uri pentru catalog și comenzi pe un dataset de minimum un milion de rânduri generate. Detectează și repară N+1, compară entity loading cu projection, atașează planurile SQL și un raport de performanță reproductibil.

Capitolul 8 — Securitate în Spring

8.1 Modelul de amenințări

Înainte de configurare, identifică:

- assets și trust boundaries;
- autentificare versus autorizare;
- user, service account și workload identity;
- least privilege și deny by default;
- input neîncrezător, SSRF, injection și deserializare;
- secrets, rotație și audit;
- date personale, retenție și redaction.

8.2 Spring Security

Stăpânește:

- **SecurityFilterChain**;
- password hashing pentru sisteme care chiar dețin parole;
- session/cookie versus bearer token;
- OAuth 2.0 și OpenID Connect;
- resource server JWT: signature, **iss**, **exp**, **nbf** și **aud** [S21];
- mapping scopes/roles și method security;
- CORS și CSRF ca probleme diferite;
- authorization tests negative;
- securizarea Actuator și a endpoint-urilor administrative.

Nu „decodezi” doar JWT-ul. Îți verifici semnătura, issuer-ul, audience-ul, timpul și algoritmiile acceptați. Nu loga token-uri sau date sensibile.

8.3 Kafka security

Învăță:

- TLS pentru transport;
- SASL mechanisms și autentificarea clienților;
- ACL/RBAC la topics și consumer groups;
- separarea credentialelor producer/consumer/admin;
- protecția Schema Registry;
- secret rotation;
- autorizarea la nivel de eveniment când topic ACL nu este suficient.

Gate 8

Protejează API-ul ca OAuth resource server, implementează autorizare pe operații și testează token expirat, issuer greșit, audience greșit și rol insuficient. Configurează separat identitățile Kafka ale producătorului și consumerului.

Capitolul 9 — Apache Kafka: modelul mental corect

9.1 Ce este Kafka

Kafka este o platformă de event streaming, nu doar o coadă. Un topic este împărțit în partiții, fiecare fiind un log ordonat. Evenimentele cu aceeași cheie ajung în aceeași partiție, iar ordinea este garantată în cadrul acelei topic-partition, nu global [S22].

Stăpânește:

- event, record key, value, headers și timestamp;
- topic, partition, offset și segment;
- leader, follower, replication factor și ISR;
- retention by time/size și log compaction;
- broker, controller și metadata quorum;
- throughput prin batching și acces secvențial;
- diferența dintre event stream și state store.

9.2 Alegerea cheii și a partițiilor

Cheia decide localitatea și ordinea. Alege cheia după invariantul ce trebuie serializat: **orderId**, **accountId**, **customerId**. O cheie constantă distruge paralelismul; o cheie random distruge ordinea per entitate.

Numărul de partiții influențează:

- paralelismul maxim al unui consumer group;
- costul metadata și al rebalansărilor;
- distribuția cheilor fierbinți;
- posibilitatea de scalare;
- ordinea la creșterea partițiilor, deoarece mapping-ul cheie-partiție se poate schimba.

Dimensionează din throughput măsurat per partition, target de headroom și număr maxim de consumers utili. Nu folosi o formulă magică.

9.3 Consumer groups și offsets

Într-un consumer group, o partiție activă este procesată de un singur membru la un moment dat; alte grupuri pot citi independent același topic [S23]. Învață:

- subscribe versus assign;

- group coordinator și assignment;
- committed offset versus current position;
- auto commit versus manual/container-managed commit;
- **auto.offset.reset** și riscul setării **latest** pentru grupuri noi;
- **max.poll.interval.ms**, **max.poll.records** și session timeout;
- static membership și cooperative rebalancing;
- pause/resume și backpressure.

Din Kafka 4.0, noul Consumer Rebalance Protocol este GA și se activează explicit prin **group.protocol=consumer**; mută heartbeat/session și assignor logic spre broker și reduce barierele globale [S24]. Învață și protocolul classic, fiindcă multe sisteme îl folosesc încă.

9.4 KRaft și operare cluster

Kafka modern folosește KRaft pentru metadata. În medii critice, documentația recomandă separarea rolurilor broker/controller și un quorum impar, uzual 3 sau 5 controllers [S25]. Ca developer trebuie să înțelegi:

- de ce majoritatea controllerilor trebuie să fie disponibilă;
- replication factor și **min.insync.replicas**;
- unclean leader election și riscul de pierdere;
- rack awareness;
- upgrade compatibility între broker și client;
- faptul că un single-broker local nu reproduce failure modes din producție.

Gate 9

Pornește un cluster local cu trei brokers. Creează topic-uri cu chei diferite, inspectează distribuția și offsets, scalează un consumer group peste numărul de partiții, oprește un consumer și un broker și explică exact ce se reatribuie și ce rămâne ordonat.

Capitolul 10 — Producer: throughput și durabilitate

10.1 Calea unui record

Învață:

- serializer, partitioner și metadata;
- accumulator, batch și sender;
- **batch.size**, **linger.ms** și compression;
- **buffer.memory** și backpressure;
- **delivery.timeout.ms**, request timeout și retries;
- callbacks și gestionarea erorilor asincrone;

- flush/close graceful.

Măsoară records/sec, bytes/sec, batch size, request latency și error rate. Un producer rapid care pierde erorile din callback nu este corect.

10.2 acks, replicare și idempotence

Cu **acks=all**, leaderul așteaptă toate replicile in-sync cerute; aceasta este garanția cea mai puternică disponibilă la nivel de ack [S26]. Leag-o de **min.insync.replicas** și replication factor.

Idempotent producer previne dublurile produse de retries în aceeași sesiune/protocol și este activ implicit când configurațiile nu intră în conflict. Necesită **acks=all**, retries pozitive și maximum cinci requests in flight per connection [S26]. Nu confunda aceasta cu idempotency end-to-end: un request HTTP repetat poate crea două evenimente business distincte dacă aplicația nu are cheie de idempotency.

10.3 Tranzacții Kafka

transactional.id activează tranzacțiile și implică idempotence. Ele pot grupa atomic mai multe writes și offsets în Kafka. Consumatorii care trebuie să ignore tranzacții abortate folosesc **isolation.level=read_committed**.

Tranzacțiile Kafka nu fac atomică, prin magie, o scriere PostgreSQL plus una Kafka. Pentru DB + Kafka folosește de regulă transactional outbox, CDC sau reconciliere; nu promite atomicitate distribuită fără protocol și infrastructură care o oferă.

Gate 10

Rulează benchmark-uri cu diferite **acks**, compression, batch și linger. Oprește brokerul leader în timpul producerii. Demonstrează ce se pierde, ce se duplică și ce se reîncearcă. Livrează configurația aleasă plus ipoteze și rezultate.

Capitolul 11 — Consumer: corectitudine, retry și reprocessing

11.1 Delivery semantics reale

- **At-most-once:** commit înainte de procesare; poate pierde efecte.
- **At-least-once:** procesare înainte de commit; poate repeta efecte.
- **Exactly-once Kafka:** proprietate pentru fluxuri read-process-write în Kafka cu transactions/read_committed; nu acoperă automat efectele externe.

Presupune duplicate la granițele cu DB, email, plăți sau API-uri. Proiectează efectul idempotent prin event ID, business key, unique constraint și tabel de inbox/deduplication.

11.2 Offset management

Învăță:

- commit sync/async și consecințe;
- record versus batch listeners;
- când offset-ul este considerat procesat;

- ordering și procesarea concurentă;
- rebalance listener;
- reset/seek și replay controlat;
- lag ca diferență de poziție, nu ca latență business completă.

O operație lentă care depășește **max.poll.interval.ms** poate scoate consumerul din grup și declanșa rebalance [S27]. Separă polling-ul de procesare cu grijă sau dimensionează batch/concurrency/timeouts coerent.

11.3 Retry și DLT

Clasifică erorile:

- transient: timeout, rate limit, indisponibilitate scurtă;
- permanent: schemă invalidă, invariant încălcat;
- necunoscută: necesită observare și limită.

Folosește exponential backoff cu jitter și număr finit de încercări. DLT nu este coș de gunoi; are owner, alertă, dashboard, retenție și procedură de triage/replay.

Blocking retry păstrează mai ușor ordinea, dar blochează partiția. Retry topics cresc disponibilitatea, dar modifică ordinea temporală. Spring Kafka avertizează explicit că non-blocking retry pierde garanția de ordering pentru topic [S28].

11.4 Poison messages

Pentru deserializare sau contract invalid:

- păstrează payload-ul brut și metadata în siguranță;
- nu loga date sensibile;
- publică într-un DLT compatibil;
- alertează pe rate, nu pe fiecare record la trafic mare;
- oferă unealtă de redrive după remediere;
- previne bucla DLT -> topic original -> DLT fără limită.

Gate 11

Implementează un consumer care scrie în PostgreSQL. Trimite același event de zece ori, provoacă timeout-uri, excepții permanente și rebalance în timpul procesării. Demonstrează efect unic, offset corect, retry finit, DLT și replay auditat.

Capitolul 12 — Spring for Apache Kafka

12.1 Producer și consumer abstractions

Stăpânește:

- **ProducerFactory**, **ConsumerFactory** și **KafkaTemplate**;
- **@KafkaListener** și listener container;

- record/batch mode;
- concurrency și relația cu partițiile;
- acknowledgments și container properties;
- serializers/deserializers și message conversion;
- headers și tracing context;
- lifecycle, pause/resume și events.

Nu folosi doar defaults. Pentru fiecare listener documentează group ID, topics, concurrency, ack mode, retry policy, DLT, schema, timeout și idempotency strategy.

12.2 Error handling

Învăță **DefaultErrorHandler**, backoff, **DeadLetterPublishingRecoverer** și **AfterRollbackProcessor**. În containere tranzacționale, o excepție provoacă rollback, iar recordul poate fi reluat conform procesorului post-rollback [S29].

Spring Kafka oferă **@RetryableTopic** și configurare pentru non-blocking retries, dar acestea nu se combină cu batch listeners sau container transactions [S28]. Alege pattern-ul pe baza ordering-ului și a efectelor, nu a comodității adnotării.

12.3 Transactions și EOS

Cu un transaction manager Kafka, containerul poate porni tranzacția înainte de listener; writes prin **KafkaTemplate** și offsets intră în aceeași tranzacție. Pentru secvența **read - process - write**, Spring Kafka oferă EOS peste Kafka, în timp ce read/process pot fi reluate la rollback [S30].

Testează producer fencing, transaction timeout și **read_committed**. Nu extinde afirmația „exactly once” la emailuri sau baze de date externe.

12.4 Testing

Spring Kafka oferă **@EmbeddedKafka** [S31]. Folosește-l pentru feedback rapid și teste concentrate. Pentru compatibilitate mai realistă, folosește Testcontainers Kafka; ghidul oficial arată integrarea cu Spring Boot, JUnit și **DynamicPropertySource** [S32].

Gate 12

Construiește o aplicație Spring Boot cu trei listeners: unul at-least-once idempotent, unul cu retry topics/DLT și unul tranzacțional Kafka read-process-write. Pentru fiecare, scrie teste cu broker real în container și o pagină care delimitează garanțiile.

Capitolul 13 — Contracte de evenimente și schema evolution

13.1 Event design

Un eveniment de integrare trebuie să exprime un fapt trecut și să aibă:

- event ID unic;
- event type și schema version;

- occurred-at și producer metadata;
- aggregate/business key;
- correlation/causation IDs;
- payload suficient, dar fără date sensibile inutile.

Distinge:

- **command:** cerere către un owner, poate fi respinsă;
- **event:** fapt deja produs;
- **notification event:** semnal minimal, cere lookup;
- **event-carried state transfer:** include starea necesară consumerului.

Evită „EntityUpdated” cu payload generic; semantica slabă mută coupling-ul în interpretare.

13.2 Formate și registry

Compară JSON, Avro și Protobuf după:

- lizibilitate și tooling;
- dimensiune și viteză;
- code generation;
- optional/default semantics;
- compatibilitate și ecosistem.

Schema Registry centralizează contractele, versiunile și verificările de compatibilitate pentru Avro, JSON Schema și Protobuf [S33]. În modul backward, un consumer cu schema nouă poate citi date scrise cu schema precedentă; transitive verifică toate versiunile, nu doar ultima [S33].

13.3 Reguli de evoluție

Practică:

- adaugă câmpuri cu default/optional conform formatului;
- nu schimba sensul unui câmp păstrând numele;
- nu reutiliza enum values eliminate;
- păstrează fixtures pentru versiuni vechi;
- verifică schema compatibility în CI;
- upgradează producerii și consumerii în ordinea cerută de modul de compatibilitate;
- pentru schimbări mari, publică un nou event type/topic și migrează gradual.

Gate 13

Definește trei versiuni ale unui eveniment **OrderConfirmed**. Scrie teste ca un consumer nou să citească payload-uri istorice și ca CI să respingă o schimbare incompatibilă. Documentează ordinea de rollout.

Capitolul 14 — Event-driven architecture și consistență

14.1 Când folosești Kafka

Kafka este potrivit pentru:

- integrare asincronă și decuplare temporală;
- fan-out către mai mulți consumatori;
- replay și audit de evenimente;
- throughput mare și procesare streaming;
- propagarea schimbărilor de stare.

Nu este automat alegerea potrivită pentru:

- request/response simplu cu rezultat imediat;
- volum mic și o singură destinație fără nevoie de replay;
- tranzacții distribuite care cer consistență sincronă strictă;
- transfer de blob-uri mari;
- workflow fără owner și fără semantică de compensare.

14.2 Transactional outbox

Problema dual write: aplicația scrie DB și publică Kafka, iar una dintre operații reușește și cealaltă eșuează.

Outbox pattern:

- 1 salvează schimbarea de business și rândul outbox în aceeași tranzacție DB;
- 2 un relay citește outbox-ul și publică în Kafka;
- 3 marchează/publică repetabil;
- 4 consumerii sunt idempotenți.

Poți implementa relay prin polling cu locking/batching sau CDC. Măsoară outbox age, backlog, publish failures și duplicate rate. Curăță/partiționează tabelul controlat.

14.3 Inbox și consumatori idempotenți

În aceeași tranzacție cu efectul local:

- inserează event ID într-un tabel cu unique constraint;
- dacă există, tratează recordul ca deja procesat;
- altfel aplică schimbarea și commit.

Nu folosi doar un cache cu TTL pentru efecte financiare: la expirare sau failover duplicatele reapar.

14.4 Saga

Pentru procese lungi:

- definește pași, owner și state machine;
- fiecare pas are outcome observabil;
- compensarea este o acțiune business, nu rollback tehnic perfect;
- timeouts și duplicate commands sunt inevitabile;
- orchestrarea centralizează controlul; choreography reduce coordonatorul, dar poate ascunde fluxul.

Documentează stările terminale, manual intervention și reconcilierea. Evită lanțurile de evenimente fără owner.

Gate 14

Implementează fluxul order - payment - inventory - shipment cu outbox și saga. Injectează eșec după fiecare pas, inclusiv după publish înainte de marcarea. Demonstrează recuperarea, compensarea, deduplicarea și reconcilierea.

Capitolul 15 — Kafka Streams și Kafka Connect

15.1 Kafka Streams

Învăță:

- KStream, KTable și GlobalKTable;
- stateless map/filter/branch;
- group, aggregate și reduce;
- tumbling, hopping, sliding și session windows;
- event time, grace și late events;
- stream-stream, stream-table și table-table joins;
- repartitioning și co-partitioning;
- Serdes;
- topology description și test driver.

15.2 State stores și fault tolerance

Processor API permite procesare custom și state stores. Store-urile persistente sunt în mod uzual RocksDB local și sunt refăcute din changelog topics compactate [S35]. Învăță:

- local state și task ownership;
- changelog și restore time;
- standby replicas;

- disk sizing;
- interactive queries;
- topology changes și application ID;
- exact-once-v2 prin **processing.guarantee** [S36].

15.3 Kafka Connect

Învăță:

- source și sink connectors;
- workers standalone/distributed;
- tasks, offsets și converters;
- Single Message Transforms;
- error tolerance și DLT;
- plugin isolation;
- securitatea credentialelor;
- CDC connectors și snapshot/streaming.

Folosește Connect pentru integrare standardizată cu sisteme, nu scrie un microserviciu bespoke fără motiv. Scrie cod custom când transformarea, protocolul sau garanțiile cer control pe care connectorul nu îl oferă.

Gate 15

Construiește un pipeline Streams care calculează vânzări pe ferestre, gestionează late events, face join cu un catalog și publică rezultate exactly-once-v2. Oprește o instanță și măsoară restore/rebalance. Adaugă un connector sink într-un mediu de test.

Capitolul 16 — Testarea unui sistem distribuit

16.1 Piramida practică

Folosește:

- multe unit tests pentru domeniu și mapping;
- slice tests pentru MVC/JPA/security;
- integration tests cu DB/Kafka/Schema Registry reale;
- contract tests pentru HTTP și evenimente;
- puține end-to-end tests pentru fluxuri critice;
- load, soak și fault tests înainte de producție.

@SpringBootTest peste tot încetinește feedback-ul și poate ascunde designul slab. Mocking KafkaTemplate verifică doar apelul, nu serializarea, routing-ul, commit-ul sau brokerul.

16.2 Testcontainers

Rulează versiuni explicite și apropiate de producție pentru PostgreSQL, Kafka și dependențe. Stăpânește:

- lifecycle per suite versus per test;
- dynamic properties;
- network aliases;
- reusable fixtures controlate;
- așteptare pe condiție, nu **sleep** arbitrar;
- logs/artifacts păstrate la failure.

16.3 Fault injection

Scenarii obligatorii:

- broker indisponibil;
- leader election;
- consumer kill înainte și după efectul DB;
- DB timeout/deadlock;
- duplicate și out-of-order records;
- payload invalid și schema incompatibilă;
- rebalance în timpul procesării;
- storage plin sau latență mare;
- rolling deployment cu două versiuni.

16.4 Performance tests

Definește workload, warm-up, durată și distribuție realistă a cheilor. Măsoară:

- throughput susținut;
- end-to-end event age;
- p50/p95/p99;
- consumer lag per partition;
- CPU, heap, allocations, GC și network;
- DB pool și query latency;
- backlog recovery rate.

Gate 16

Creează o suită reproductibilă care pornește infrastructura, generează workload, injectează trei defecte și produce un raport. Testul trebuie să distingă pierderea, duplicarea, întârzierea și indisponibilitatea.

Capitolul 17 — Observabilitate și operare

17.1 Logs, metrics, traces

Spring Boot Actuator oferă health, management și integrare Micrometer [S37]. Observability înseamnă logging, metrics și traces corelate prin Micrometer Observation/OpenTelemetry [S38][S39].

Logs:

- JSON structurat;
- timestamp UTC, level, service, version și environment;
- trace/correlation/event ID;
- fără token-uri, parole sau payload sensibil;
- rate limiting/sampling pentru erori repetitive.

Metrics:

- rate, errors, duration și saturation;
- low-cardinality labels;
- business outcomes;
- histograms pentru percentiles utile;
- fără user ID/order ID ca label.

Traces:

- propagă contextul de la HTTP la DB, Kafka și consumer;
- păstrează correlation și causation IDs în evenimente;
- adaugă spans custom doar pentru pași care ajută diagnosticul;
- stabilește sampling policy.

17.2 Metrici Kafka

Documentația Kafka recomandă monitorizarea ratelor de mesaje/bytes, request latency, consumer max lag și fetch rate [S40]. Urmărește:

- under-replicated/offline partitions;
- ISR shrink/expand;
- controller/quorum health;
- produce/fetch request latency și errors;
- bytes/records in/out;
- consumer lag și lag growth;
- rebalance count/duration;
- DLT și retry rate;

- outbox oldest age;
- end-to-end event age.

Lag 100.000 poate fi acceptabil sau critic în funcție de rata de recuperare și SLO. Leagă alerta de impact, growth și timp până la încălcarea SLO.

17.3 SLI, SLO și alerte

Exemple:

- 99,9% dintre comenzile acceptate ajung în starea confirmată sau compensată în 2 minute;
- 99% dintre evenimente sunt procesate end-to-end în 10 secunde;
- rata de evenimente în DLT sub 0,01%;
- zero efecte financiare duplicate.

Alertele trebuie să fie acționabile și să trimită la dashboard/runbook. Separă warning-ul de page. Evită alertele pe CPU fără impact observabil.

17.4 Runbook și incidente

Pentru fiecare flux critic documentează:

- cum confirmi impactul;
- dashboard-uri și query-uri;
- cum oprești/pauzezi consumerul;
- cum scalezi și care este limita;
- cum inspecțezi DLT/outbox;
- cum faci offset reset/replay în siguranță;
- rollback și escaladare;
- cum verifici recuperarea.

Gate 17

Adaugă Actuator, Prometheus/OpenTelemetry și dashboard-uri. Simulează lag, DLT spike și DB pool exhaustion. Alerta trebuie să identifice fluxul, severitatea și primul pas din runbook.

Capitolul 18 — Resilience și integrarea cu servicii externe

18.1 Timeouts, retries și circuit breakers

Fiecare apel extern are timeout. Retry doar pentru operații tranzitorii și idempotente, cu backoff și jitter. Un circuit breaker limitează presiunea asupra unei dependențe bolnave; bulkhead-ul limitează contaminarea resurselor.

Învăță:

- deadline budget propagat;

- retry amplification între straturi;
- circuit breaker states;
- rate limiter;
- concurrency limiter/bulkhead;
- fallback explicit și date stale marcate;
- hedging doar în cazuri măsurate.

Nu combina trei retries în client, service și Kafka fără calcul: poți produce zeci de apeluri pentru un singur eveniment.

18.2 Backpressure

Kafka poate absorbi backlog, dar DB/API-ul downstream are limită. Controlează:

- consumer concurrency;
- batch size;
- pause/resume;
- connection pool;
- in-flight requests;
- rate limiting;
- autoscaling după lag și capacitate reală.

Gate 18

Integrează un API instabil. Configurează deadline, circuit breaker și rate limit. Rulează un test în care dependența are 50% timeouts și demonstrează că aplicația nu epuizează threads/connections și recuperează backlog-ul controlat.

Capitolul 19 — Containers, Kubernetes și CI/CD

19.1 Docker

Învăță:

- multi-stage builds;
- imagini JRE/minimal și pinning;
- non-root user și read-only filesystem unde posibil;
- JVM container limits;
- layers și build cache;
- SBOM, image scanning și signing;
- configurare la runtime;
- shutdown cu SIGTERM și închiderea consumerilor.

19.2 Kubernetes

Stăpânește:

- Deployment, Service, ConfigMap și Secret;
- requests/limits și QoS;
- startup, readiness și liveness probes;
- rolling update, PodDisruptionBudget și anti-affinity;
- graceful termination și **terminationGracePeriodSeconds**;
- HPA și metrice custom;
- network policies și service accounts.

Kubernetes folosește readiness pentru a opri traficul către un pod nepregătit și liveness pentru restart; o liveness greșită poate produce cascading failures [S41]. Pentru Kafka consumers, readiness nu trebuie să declare „mort” procesul doar fiindcă lag-ul a crescut temporar.

19.3 Pipeline sigur

Un pipeline matur:

- 1 compilează și rulează verificări statice;
- 2 rulează unit/integration/contract tests;
- 3 verifică schema compatibility;
- 4 construiește și scanează imaginea;
- 5 aplică migrarea expand;
- 6 deploy gradual/canary;
- 7 verifică SLO și smoke tests;
- 8 promovează sau rollback;
- 9 face ulterior cleanup/contract migration.

Gate 19

Deployează proiectul pe un cluster local sau cloud de test. Fă rolling update în timp ce producerii trimit trafic. Demonstrează compatibilitatea între versiuni, graceful shutdown, lipsa unui rebalance storm și rollback-ul.

Capitolul 20 — System design și relația cu business-ul

20.1 Discovery

Întrebări obligatorii:

- ce rezultat business urmărim și cine îl deține?
- ce SLO și volum avem azi și peste un an?

- ce consistență este obligatorie?
- ordinea trebuie păstrată pentru ce entitate?
- ce se întâmplă la duplicate, întârziere sau indisponibilitate?
- cât timp păstrăm/reprocesăm datele?
- cine deține schema și topic-ul?
- ce date sunt sensibile?
- care este procesul manual de fallback?

Transformă „vrem Kafka ca să scalăm” în cifre: events/sec, bytes/event, peak factor, număr de consumers, latență, retenție, disponibilitate și cost.

20.2 Documente de design

Un design review bun conține:

- context și non-goals;
- requirements funcționale și de calitate;
- volum și ipoteze;
- diagramă de context și sequence pentru happy/failure paths;
- data model și contracte;
- consistency/delivery semantics;
- threat model;
- observability și SLO;
- rollout, migration, rollback și cost;
- alternative respinse și consecințe.

20.3 Estimare și comunicare

Separă:

- necunoscute tehnice de efortul repetabil;
- POC de production readiness;
- throughput de latență;
- „exactly once în Kafka” de efect unic în business;
- disponibilitate de corectitudine.

Spune clar: „putem face X dacă acceptăm Y; pentru Z avem nevoie de mecanismul W și de aceste teste”. Asta inspiră mai multă încredere decât un „da” fără limite.

Gate 20

Scrie și prezintă un design pentru o platformă de comenzi cu 10.000 events/sec, două regiuni și retenție de șapte zile. Include calcule, failure modes, RPO/RTO, cost drivers și plan de migrare. Apăra-l într-o sesiune de întrebări.

Planul de 40 de săptămâni

Faza 1 — Java și engineering discipline (săptămânile 1–6)

Săptămâna 1 — Java core

- records, sealed types, equality, exceptions;
- kata de modelare a banilor și comenzilor;
- gate: invariants și teste fără framework.

Săptămâna 2 — Collections, generics, streams

- complexitate și alegerea structurii;
- generics/wildcards;
- benchmark pentru două implementări.

Săptămâna 3 — Build și testare

- Maven/Gradle, JUnit, AssertJ, Mockito;
- CI și separarea testelor;
- gate: build reproductibil.

Săptămâna 4 — Design modular

- hexagonal, package by feature, ADR;
- domeniu independent de Spring;
- review de coupling.

Săptămâna 5 — Concurență

- JMM, locks, atomics, executors, cancellation;
- reproduce și repară un race condition.

Săptămâna 6 — JVM și profiling

- heap, GC, threads, JFR;
- raport before/after pentru o problemă reală.

Faza 2 — Spring Boot și date (săptămânile 7–14)

Săptămâna 7 — Spring Core

- DI, lifecycle, configuration, proxies;
- context mic și clase testabile direct.

Săptămâna 8 — Spring Boot

- auto-configuration, properties, profiles, Actuator;
- diagnostic al unui bean/config failure.

Săptămâna 9 — REST

- HTTP, DTO, validation, ProblemDetail, OpenAPI;
- API cu contract și erori coerente.

Săptămâna 10 — SQL/PostgreSQL

- modelare, joins, indexes, query plans;
- optimizarea unui query pe volum.

Săptămâna 11 — Tranzacții

- isolation, locking, deadlocks;
- test concurent reproductibil.

Săptămâna 12 — JPA/Hibernate

- lifecycle, mappings, flush, fetching;
- detectare și reparare N+1.

Săptămâna 13 — Spring Data și migrații

- projections, repositories, Flyway;
- expand/backfill/contract.

Săptămâna 14 — Security

- OAuth resource server, JWT, authorization;
- teste negative și threat model.

Faza 3 — Kafka core (săptămânile 15–23)

Săptămâna 15 — Log, topics și partitions

- retention, compaction, replication;
- experimente cu chei și ordering.

Săptămâna 16 — Producers

- batching, compression, acks, idempotence;
- benchmark și failure injection.

Săptămâna 17 — Consumers și groups

- poll loop, offsets, assignments;
- scale și rebalance controlat.

Săptămâna 18 — Delivery semantics

- at-most/at-least/exactly-once;
- matrice de garanții și efecte externe.

Săptămâna 19 — Retry și DLT

- clasificare erori, backoff, poison records;
- operator flow de redrive.

Săptămâna 20 — Spring Kafka

- template, listeners, containers, ack modes;
- aplicație cu politici explicite.

Săptămâna 21 — Kafka transactions

- transactional producer, read_committed, EOS;
- rollback/fencing tests.

Săptămâna 22 — Schemas

- Avro/Protobuf/JSON Schema și registry;
- compatibility gate în CI.

Săptămâna 23 — Kafka operations

- KRaft, replication, upgrades și metrics;
- broker failure și diagnostic.

Faza 4 — Arhitectură distribuită (săptămânile 24–30)

Săptămâna 24 — Event design

- commands versus events, ownership și headers;
- review al contractelor.

Săptămâna 25 — Outbox/inbox

- dual-write, relay, CDC și dedup;
- crash în fiecare punct critic.

Săptămâna 26 — Saga

- state machine, compensation și reconciliation;
- flux cu intervenție manuală.

Săptămâna 27 — Kafka Streams I

- KStream/KTable, aggregate, windows;
- topologie cu event time.

Săptămâna 28 — Kafka Streams II

- joins, state stores, restore, EOS-v2;
- failover și măsurare restore.

Săptămâna 29 — Kafka Connect

- source/sink, offsets, converters, CDC;
- configurează un connector real.

Săptămâna 30 — Resilience

- timeouts, circuit breaker, bulkhead, backpressure;

- recovery sub dependență degradată.

Faza 5 — Production engineering (săptămânile 31–36)

Săptămâna 31 — Test strategy

- pyramid, slices, Testcontainers, contract tests;
- suită rapidă plus suite realistă.

Săptămâna 32 — Load și chaos

- workload, percentiles, soak, failure injection;
- raport reproductibil.

Săptămâna 33 — Observability

- logs, metrics, traces și correlation;
- dashboard end-to-end.

Săptămâna 34 — SLO și incidente

- SLI, alerts, runbooks și postmortem;
- game day cu trei incidente.

Săptămâna 35 — Docker/Kubernetes

- image hardening, probes, graceful shutdown;
- rolling deployment sub trafic.

Săptămâna 36 — CI/CD și security hardening

- schema gate, scanning, canary, rollback;
- threat model actualizat.

Faza 6 — Portofoliu și interviu (săptămânile 37–40)

Săptămâna 37 — Proiect final: domeniu și design

- requirements, SLO, ADR, schemas și plan de test;
- design review înainte de implementarea finală.

Săptămâna 38 — Proiect final: reliability

- outbox/inbox, retries, DLT, replay;
- failure matrix executată.

Săptămâna 39 — Proiect final: production

- observability, performance, deployment și runbook;
- demo sub trafic și defecte.

Săptămâna 40 — Ambalare profesională

- README, diagrame, video demo și postmortem;

- mock system design și behavioral interview;
- CV bazat pe rezultate măsurabile.

Proiecte de portofoliu

Proiectul A — Platformă de comenzi și plăți

Servicii/module: order, inventory, payment, shipment.

Obligatoriu:

- Spring Boot, PostgreSQL și Flyway;
- OAuth resource server;
- outbox și consumers idempotenți;
- saga cu compensation;
- Avro/Protobuf plus schema compatibility;
- retry/DLT și redrive;
- Testcontainers;
- metrics/traces și SLO;
- Kubernetes rolling deployment.

Scenarii demonstrate: request duplicat, payment timeout, inventory conflict, producer retry, consumer crash după DB commit, broker failover și replay.

Proiectul B — Documente și notificări în timp real

Flux: upload metadata - scan - classify - notify - audit.

Obligatoriu:

- API idempotent;
- blob-ul în object storage, nu în Kafka;
- evenimente cu correlation/causation IDs;
- fan-out către email, webhook și audit;
- rate limit/bulkhead per provider;
- retry topics și DLT;
- tenant isolation și redaction;
- dashboard pentru event age și failures.

Proiectul C — Fraud/analytics cu Kafka Streams

Obligatoriu:

- event-time windows;
- stream-table join cu profilul clientului;

- dedup state store;
- late-event policy;
- exactly-once-v2;
- state restore și standby;
- load test cu hot keys;
- query/read model pentru investigații.

Cum arată un repository convingător

- README cu problema și rezultatul, nu doar comenzi de pornire;
- diagramă context, container și două sequence diagrams;
- ADR-uri pentru Kafka, keying, outbox și schema format;
- **docker compose** pentru demo local;
- teste care reproduc failures;
- dashboard screenshots;
- benchmark cu metodologie;
- runbook și postmortem;
- secțiune „guarantees and non-guarantees”.

Checklist de competență

Java/JVM

- [] Explic JMM și happens-before printr-un exemplu.
- [] Aleg colecția și complexitatea potrivită.
- [] Folosesc virtual threads doar pentru workload potrivit.
- [] Diagnostichez CPU, allocation, locks și GC cu JFR.
- [] Scriu cod testabil fără container Spring.

Spring și date

- [] Înțeleg DI, proxies și **@Transactional** boundaries.
- [] Proiectez DTO, validation și Problem Details.
- [] Securizez un resource server și testez autorizarea negativă.
- [] Detectez N+1 și citesc query plans.
- [] Fac migrații expand/contract fără downtime inutil.

Kafka

- [] Explic topic, partition, offset, ISR și consumer group.
- [] Aleg cheia după invariantul de ordering.

- [] Configurez producer durability și idempotence conștient.
- [] Proiectez consumers idempotenți și offset policy.
- [] Delimitiez exact ce acoperă EOS.
- [] Operez retry, DLT și replay.
- [] Evoluez schema compatibil și verific în CI.
- [] Diagnostichez lag, hot partitions și rebalances.

Producție

- [] Am tests cu infrastructură reală în containere.
- [] Am SLO, alerts și runbooks.
- [] Propag trace/correlation prin Kafka.
- [] Fac graceful shutdown și rolling deploy.
- [] Pot conduce un incident și scrie un postmortem fără vină.

Întrebări de interviu pe care trebuie să le poți apăra

- 1 Ce se întâmplă dacă producerul primește timeout după ce brokerul a scris recordul?
- 2 De ce idempotent producer nu face idempotent un payment API?
- 3 Cum alegi cheia și numărul de partiții?
- 4 Ce se întâmplă când ai mai mulți consumers decât partiții?
- 5 Când poate apărea un rebalance și cum îl reduci?
- 6 Ce înseamnă **read_committed**?
- 7 Cum rezolvi dual write între PostgreSQL și Kafka?
- 8 Cum faci replay fără efecte duble?
- 9 Blocking retry sau retry topics: ce pierzi și ce câștigi?
- 10 Ce metrici disting un consumer lent de unul blocat?
- 11 Cum evoluezi un eveniment folosit de zece echipe?
- 12 De ce un **@Transactional** apelat prin self-invocation poate surprinde?
- 13 Cum identifici N+1 și cum dovedești remedierea?
- 14 Când alegi virtual threads și când nu?
- 15 Cum proiectezi probes pentru un Kafka consumer?
- 16 Ce faci dacă DLT-ul crește, dar serviciul pare „healthy”?
- 17 Cum rulezi o migrare DB compatibilă cu două versiuni ale aplicației?
- 18 Modular monolith sau microservices: ce dovezi cer înainte de separare?

19 Cum urmărești aceeași comandă prin HTTP, DB, Kafka și trei consumers?

20 Ce afirmații poți face onest despre „exactly once” în designul tău?

Greșeli frecvente

- învățarea adnotărilor fără HTTP, SQL, JVM și failure modes;
- entități JPA expuse direct în API;
- tranzații lungi care includ rețea;
- **@SpringBootTest** și mocks peste tot;
- cheie Kafka aleasă random sau absentă fără analiză;
- „mai multe partiții” ca răspuns universal;
- auto-commit fără înțelegerea momentului;
- retry infinit și fără jitter;
- DLT fără owner și redrive;
- schema JSON neverificată;
- promisiuni exactly-once peste efecte externe;
- lag folosit ca singura metrică;
- liveness dependentă de toate downstream-urile;
- autoscaling peste capacitatea DB;
- microservicii înainte de limite modulare clare;
- lipsa unei proceduri de replay și reconciliation.

Bibliografie primară și traseu de lectură

Java și JVM

[S1] OpenJDK, JDK 25 — <https://openjdk.org/projects/jdk/25/>

[S2] Oracle Java 25, Collections Framework — <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/doc-files/coll-index.html>

[S3] Oracle Java 25, **java.util.concurrent** — <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/package-summary.html>

[S4] OpenJDK JEP 444, Virtual Threads — <https://openjdk.org/jeps/444>

[S5] OpenJDK JEP 453, Structured Concurrency — <https://openjdk.org/jeps/453>

[S6] Oracle Java 25, Garbage Collection Tuning Guide — <https://docs.oracle.com/en/java/javase/25/gctuning/index.html>

[S7] Oracle Java 25, Diagnostic Tools și JFR — <https://docs.oracle.com/en/java/javase/25/troubleshoot/diagnostic-tools.html>

Spring Boot, web, date și securitate

[S8] Spring Framework, Dependency Injection — <https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html>

[S9] Spring Boot 4.1, System Requirements — <https://docs.spring.io/spring-boot/system-requirements.html>

[S10] Spring Modulith, Fundamentals — <https://docs.spring.io/spring-modulith/reference/fundamentals.html>

[S11] Spring Modulith, Module Testing — <https://docs.spring.io/spring-modulith/reference/testing.html>

[S12] Spring Framework, Validation — <https://docs.spring.io/spring-framework/reference/core/validation.html>

[S13] Spring MVC, Controller Advice — <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-advice.html>

[S14] Spring MVC, RFC 9457 Error Responses — <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-rest-exceptions.html>

[S15] PostgreSQL, Examining Index Usage — <https://www.postgresql.org/docs/current/indexes-examine.html>

[S16] Flyway, Versioned Migrations — <https://documentation.red-gate.com/fd/versioned-migrations-273973333.html>

[S17] Jakarta Persistence Specification — <https://jakarta.ee/specifications/persistence/>

[S18] Hibernate ORM User Guide — https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate_User_Guide.html

[S19] Spring Data JPA, Projections — <https://docs.spring.io/spring-data/jpa/reference/repositories/projections.html>

[S20] Spring Framework, Transaction Management — <https://docs.spring.io/spring-framework/reference/data-access/transaction.html>

[S21] Spring Security, OAuth 2 Resource Server JWT — <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>

Apache Kafka și Spring Kafka

[S22] Apache Kafka, Introduction — <https://kafka.apache.org/documentation/>

[S23] Apache Kafka, KafkaConsumer API — <https://kafka.apache.org/43/javadoc/org/apache/kafka/clients/consumer/KafkaConsumer.html>

[S24] Apache Kafka, Consumer Rebalance Protocol — <https://kafka.apache.org/43/operations/consumer-rebalance-protocol/>

[S25] Apache Kafka, KRaft — <https://kafka.apache.org/43/operations/kraft/>

[S26] Apache Kafka, Producer Configurations — <https://kafka.apache.org/43/configuration/producer-configs/>

[S27] Apache Kafka, Consumer Configurations — <https://kafka.apache.org/43/configuration/consumer-configs/>

[S28] Spring for Apache Kafka, Non-Blocking Retries — <https://docs.spring.io/spring-kafka/reference/retrytopic.html>

[S29] Spring for Apache Kafka, Handling Exceptions — <https://docs.spring.io/spring-kafka/reference/kafka/annotation-error-handling.html>

[S30] Spring for Apache Kafka, Exactly Once Semantics — <https://docs.spring.io/spring-kafka/reference/kafka/exactly-once.html>

[S31] Spring for Apache Kafka, Testing Applications — <https://docs.spring.io/spring-kafka/reference/testing.html>

[S32] Testcontainers, Testing a Spring Boot Kafka Listener — <https://testcontainers.com/guides/testing-spring-boot-kafka-listener-using-testcontainers/>

[S33] Confluent Schema Registry, Schema Evolution — <https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html>

[S34] JUnit, Current User Guide — <https://docs.junit.org/current/user-guide/>

[S35] Apache Kafka Streams, Processor API și State Stores — <https://kafka.apache.org/43/streams/developer-guide/processor-api/>

[S36] Apache Kafka, Kafka Streams Configurations — <https://kafka.apache.org/43/configuration/kafka-streams-configs/>

Producție și observabilitate

[S37] Spring Boot, Production-ready Features — <https://docs.spring.io/spring-boot/reference/actuator/index.html>

[S38] Spring Boot, Observability — <https://docs.spring.io/spring-boot/reference/actuator/observability.html>

[S39] OpenTelemetry Java — <https://opentelemetry.io/docs/languages/java/>

[S40] Apache Kafka, Monitoring — <https://kafka.apache.org/43/operations/monitoring/>

[S41] Kubernetes, Liveness, Readiness and Startup Probes — <https://kubernetes.io/docs/concepts/workloads/pods/probes/>

[S42] Spring Boot, Metrics — <https://docs.spring.io/spring-boot/reference/actuator/metrics.html>

[S43] Spring Data JPA Reference — <https://docs.spring.io/spring-data/jpa/reference/jpa.html>

[S44] Jakarta Persistence 3.2 — <https://jakarta.ee/specifications/persistence/3.2/>

Ordine recomandată a lecturii

- 1 Java/OpenJDK: [S1]–[S7].
- 2 Spring, web, data, security: [S8]–[S21], [S43]–[S44].
- 3 Kafka core: [S22]–[S27].
- 4 Spring Kafka și testare: [S28]–[S34].
- 5 Streams și producție: [S35]–[S42].

Nu încerca să citești toată documentația înainte să scrii cod. Pentru fiecare bloc: citește secțiunea relevantă, implementează un experiment, defectează-l intenționat, măsoară și scrie concluzia.

Concluzie

Un inginer Java + Spring Boot + Kafka valoros nu este persoana care cunoaște cele mai multe adnotări. Este persoana care poate explica și demonstra comportamentul sistemului când totul merge, când un pas eșuează și când o operație se repetă.

Ordinea sănătoasă este:

- 1 Java și SQL solide;
- 2 Spring folosit ca instrument de compoziție;
- 3 Kafka înțeles ca log partiționat și replicat;
- 4 corectitudine prin tranzacții locale, idempotency, outbox și contracte;
- 5 testare cu infrastructură reală și failure injection;
- 6 observabilitate, SLO, deployment și operare.

Dacă finalizezi gate-urile, cele trei proiecte și poți apăra garanțiile fără formulări vagi, ai un profil care poate duce un serviciu de la cerință la producție, nu doar de la tutorial la demo.